

DPvz and DPvzVtk Design

Douglas Pase, PhD CS&E

dmpase@sandia.gov

DPvz Files

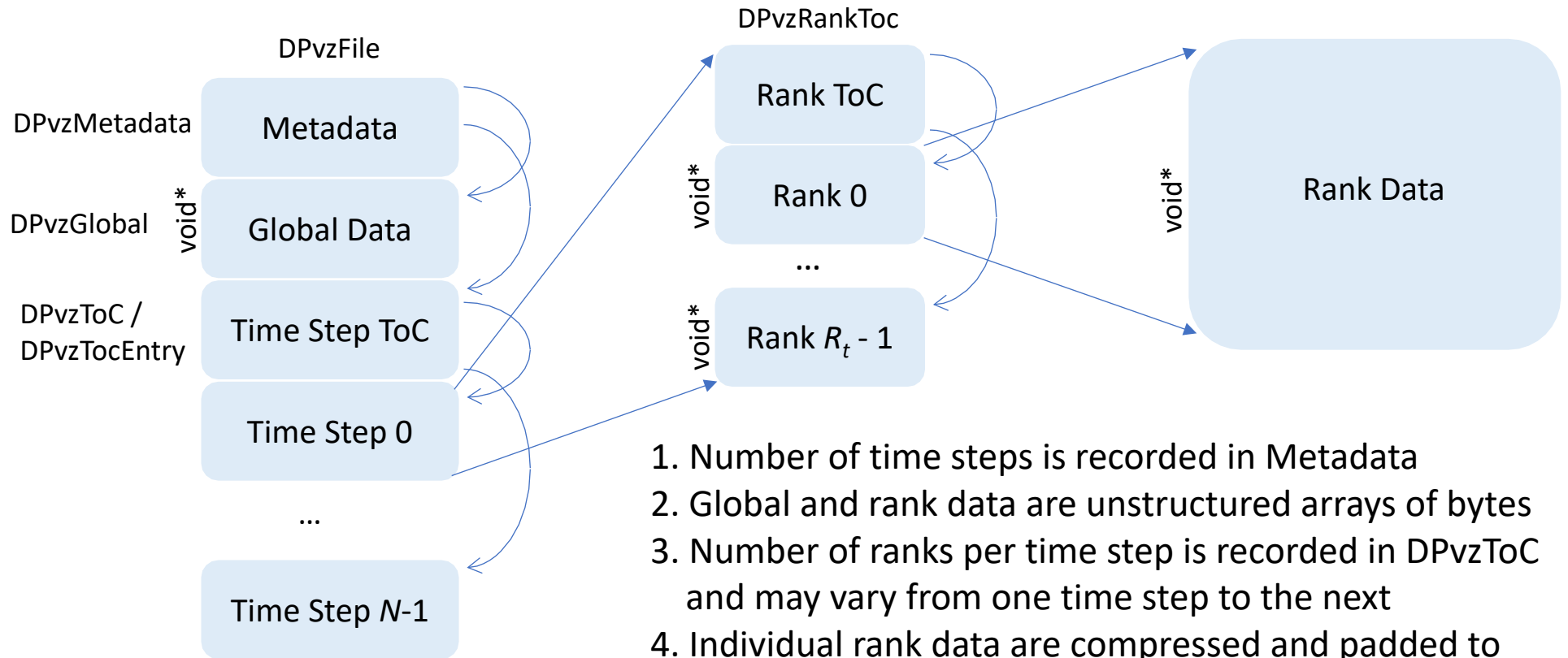
What Problem Does DPvz Solve?

- Details vary between file types, but many applications use designs that write one (or several) visualization file(s) for each MPI rank
- As simulation codes improve, simulations become larger and scale up the number of MPI ranks they use, increasing the number of files
- Opening large numbers of files simultaneously can overwhelm a file system metadata server, cratering application performance and causing temporary loss of access to the file system across the lab
- DPvz consolidates multiple files into a single parallel file, significantly reducing metadata server load, compressing data to reduce volume, and vastly simplifying file management

Introduction to DPvz

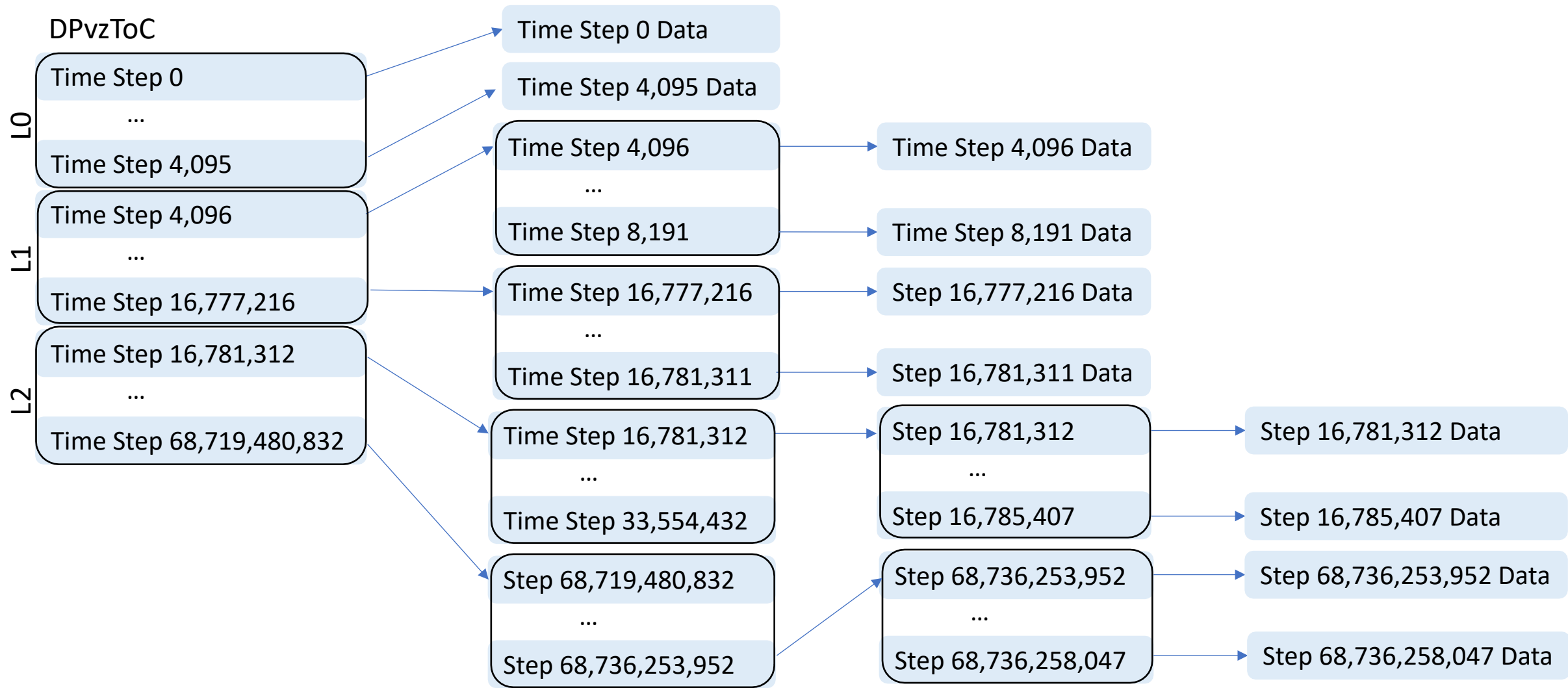
- **D**istributed **P**arallel **v**isualization with lib**z** compression
- Creates files designed to be safe for large-scale parallel r/w access
- Intended to be used as a superclass for derived file types such as DPVTK
- Organized into two levels:
 - Each file contains 0 or more time steps, identified by cycle number and time
 - Cycle and time must be monotonic, but they may vary in their separation
 - Each time step contains data from R ranks
 - Rank data is a compressed buffer of unstructured bytes
- The number of ranks may vary from one time step to the next
- The amount of data stored may vary from one rank to the next

DPvzFile High-Level Design



1. Number of time steps is recorded in Metadata
2. Global and rank data are unstructured arrays of bytes
3. Number of ranks per time step is recorded in DPvzToC and may vary from one time step to the next
4. Individual rank data are compressed and padded to align on page boundaries, to avoid false sharing of data
5. Inflated/deflated sizes and CRC of deflated data are stored in Rank ToC

DPvz Time Step Table of Contents



File Distribution In Parallel File Systems

- DPvz files are potentially very large files utilizing a high degree of parallel read and write access
- File design must avoid three problems:
 - False sharing (disjoint but adjacent parallel writes resulting in file corruption)
 - Metadata target (MDT) floods (overwhelming the metadata server)
 - Object storage target (OST) floods (overwhelming a data server)
- Disk writes use block I/O, so two ranks on different nodes that write to different data in the same block can cause data loss (false sharing)
- MDT floods are caused by large numbers of ranks opening different files, causing the file system metadata server to be overwhelmed with requests
- OST floods are caused by many ranks simultaneously writing data to a file that is allocated to a single disk subsystem, overwhelming the subsystem

File Distribution Mechanisms

- In Lustre, file distribution cannot be changed once a file is created
- DPvz provides the following functions:
 - `set_file_lfs_stripe_size` creates a new distributed file
 - `set_dir_lfs_stripe_size` sets the default distribution for a directory
 - `chk_directory_data_distribution` checks whether to set directory distributions
 - `chk_file_data_distribution` checks whether to set file distributions
- If used on a non-Lustre file system, set functions are null operations with no effect
- Caution must be used because the set functions use `fork/exec`, which causes problems for some MPI libraries and for most performance tools
 - `fork/exec` to `lfs` is the **only** mechanism available to set file distributions in Lustre
 - An API existed historically, but Lustre abandoned it several years ago
 - When used with sensitive MPI libraries, `fork/exec` must be called **before** `MPI_Init`
 - Recommend setting distributions by default to avoid user error, but allowing users to bypass the `fork/exec` when needed, requiring the file distribution to be set manually

Opening a DPvz File

- Opening a DPvz file requires the following parameters:
 - File name, with or without a file extension
 - File mode, namely, `DPvzCreate`, `DPvzReadWrite`, `DPvzReadOnly`, and `DPvzReplace`
 - Global data size
 - Magic string, e.g., “`DPvzVtk`”, to identify whether a file is of the correct type
 - File extension, e.g., “.dpv`tk`”, also used in determining file type
 - Boolean repair flag for future use in repairing damaged files
- Opening a DPvz file in an MPI application also requires:
 - MPI communicator
 - Collective operation root, which is rank 0 by default

DPvz File Mode

- `DPvzReadOnly` - open for read only, the file must exist
- `DPvzReadWrite` - open for read and write, the file must exist
- `DPvzCreate` - open for read and write, create a new file if the file doesn't exist, use the existing file if it does already exist
- `DPvzReplace` - open for read and write, create a new file if one doesn't exist, truncate and replace the file if it does exist

Global Data

- Global data is a static area of unstructured bytes
- The size of the area is specified at file creation, and is rounded up to the nearest page size (DPVZ_PAGE_SIZE)
 - When opening a file that already exists, the size parameter is ignored
- Global data is accessed using the functions:

```
int64_t DPvzFile::get_global_size()
bool    DPvzFile::get_global_data()
bool    DPvzFile::set_global_data()
```

Writing a New Time Step

- Write a new time step to a file

```
dpvz->write(cycle, time, buf, size, root);
```

- This is a collective operation; all ranks must participate
- Each rank may have a different data size, including zero bytes
- This new time step becomes the last time step in the file
 - Any time step with a cycle greater than or equal to `cycle` is deleted
 - Any time step with a time greater than or equal to `time` is deleted

Reading the Time Step Table of Contents

- Reading the number of time steps in an existing file:

```
DPvzFile* dpvz = new DPvzFile(...);  
int64_t steps = dpvz->get_steps();
```

- Reading the time step table of contents

```
DPvzTocEntry* map = new DPvzTocEntry[steps];  
dpvz->get_map(map); // 0 on success, -1 on failure
```

Reading the Table of Contents for Time Step t

- Read the number of ranks for time step t

```
int32_t ranks = map[t].ranks;
```

- Read the table of contents

```
DPvzRankToc* step_toc = new DPvzRankToc[ranks];  
dpvz->get_step_toc(map[t], step_toc); // 0 success, -1 fail
```

- DPvzRankToc contains the following information:

```
int64_t  inflated_size;    // inflated (uncompressed) data size  
int64_t  deflated_size;    // deflated (compressed), unpadded data size  
uint64_t deflated_crc;     // checksum of deflated (compressed) data  
int64_t  offset;           // starting offset of rank data
```

Reading the Data for Time Step t and Rank r

- Read the raw (inflated) data for time step t and rank r

```
char* raw = new char[step_toc[r].inflated_size];  
dpvz->get_data(step_toc[r], raw);
```

- `raw` contains the original (uncompressed) data written by rank r
- `get_data` returns 0 on success, -1 on failure

Truncating a File

- Truncate a file from a specified index or cycle and time to the file end

- Truncate based on time step index

```
dpvz->truncate(idx, root);
```

- Truncate based on cycle and time

```
dpvz->truncate(cycle, time, root);
```

- Truncate based on cycle only

```
dpvz->truncate(cycle, DBL_MAX, root);
```

- Truncate based on time only

```
dpvz->truncate(INT_MAX, time, root);
```

- Truncate the entire file

```
dpvz->truncate(0, root);
```

```
dpvz->truncate(INT_MIN, DBL_MIN, root);
```


Closing a File

- To close a file

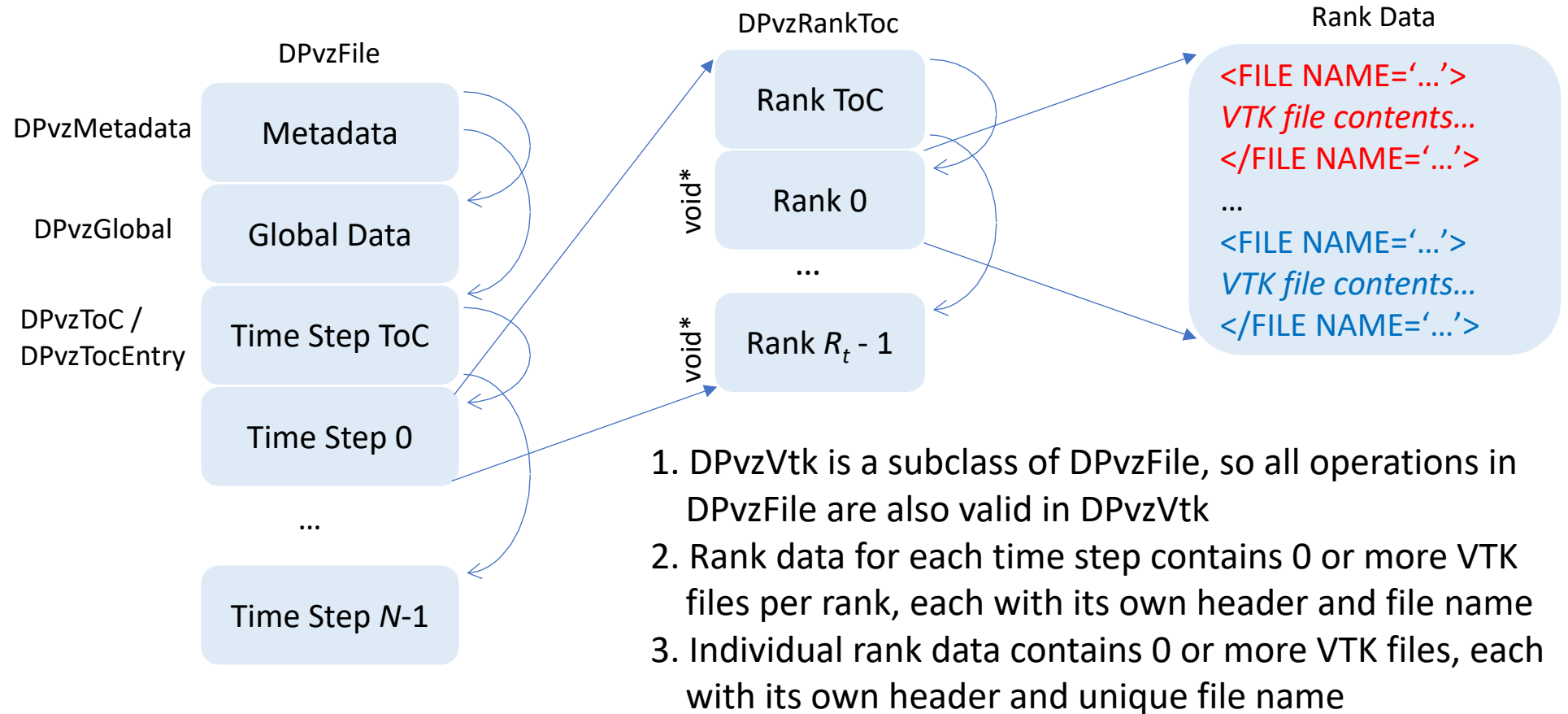
```
delete dpvz;
```

DPvzVtk Files

Notes

- What we label here as "VTK files" are just files stored in an archive
- The original use case was to store VTK files created by many ranks in a single distributed archive, hence the naming of the class
- However, the mechanism used is general and not specific to VTK
 - DPvzVtk does not look the data stored within a file, it's just raw bytes
 - It can be used to store any type of file, not just VTK files

DPvzVtk File High-Level Design



Opening a DPVTK File for Read

- DPVTK files may be opened by a serial or MPI program

- To open a file in a serial application:

```
DPvzVtk* dpvz = new DPvzVtk(file, DPvzReadOnly);
```

- To open a file in an MPI application:

```
DPvzVtk* dpvz = new DPvzVtk(file, DPvzReadOnly, MPI_COMM_WORLD);
```

- The serial and MPI versions are compiled separately, to avoid the serial version having a dependence on an MPI library that it doesn't use

Note: The MPI version is a collective operation and all ranks must participate.

Reading the DPVTK Table of Contents

- Reading the number of time steps in a file:

```
int64_t time_steps = dpvz->get_steps();
```

- Reading the DPVTK table of contents:

```
DPvzTocEntry* map = new DPvzTocEntry[time_steps];
```

```
int rv = dpvz->get_map(map); // returns 0 on success, -1 on failure
```

- `get_map` fills in the array `map` with the table of contents for the file

- Each entry in `map` points to one time step and contains:

```
int32_t  ranks;    // number of ranks participating in this time step
int32_t  cycle;    // simulation cycle of this time step
double   time;     // simulation time of this time step
uint64_t toc_crc;  // CRC64 value for the time step table of contents
int64_t  offset;   // file offset to the start of the time step data
```

Reading the Table of Contents for Time Step t

- Do the following to read the table of contents for time step t

```
int32_t ranks = map[t].ranks;  
DPvzRankToc* step_toc = new DPvzRankToc[ranks];  
dpvz->get_step_toc(map[t], step_toc);
```

- DPvzRankToc contains the following information:

```
int64_t  inflated_size;    // inflated (uncompressed) data size  
int64_t  deflated_size;    // deflated (compressed), unpadded data size  
uint64_t deflated_crc;     // checksum of deflated (compressed) data  
int64_t  offset;          // starting offset of rank data
```

- get_step_toc returns 0 on success, -1 on failure

Get VTK Files for Time Step t and Rank r

- Read the raw (inflated) data for time step t and rank r

```
char* raw = new char[step_toc[r].inflated_size];  
dpvz->get_data(step_toc[r], raw);
```

- Extract the individual VTK files from the raw data

```
int cnt = 0;  
DPvzVtkData* data = NULL;  
DPvzVtkData::extract(raw, step_toc[r].inflated_size, cnt, data);
```

- `get_data` and `extract` return 0 on success, -1 on failure
- Upon completion, `data` contains an array of `cnt` VTK files

DPvzVtkData

- DPvzVtkData contains the following information

```
char*    name;    // original name of the file, including any directory structure
char*    path;    // directory containing the file, or "./" if name has no '/'
char*    file;    // file base name, i.e., the file without the directory
int32_t  size;    // number of bytes contained in the file, i.e., in the data area
char*    data;    // contents of the file
```

Notes on Writing to a DPvzVtk File

- **DPvzVtk does not understand the contents of a VTK file**
- DPvzVtk only provides a parallel container in which to store VTK files
- Within a rank, VTK files are stored sequentially as a byte buffer
- It is the **writer's** responsibility to provide the separating header and footer in the output buffer with **exactly** the correct syntax:

```
<FILE NAME= 'VTK file name 1'>\n
```

```
VTK file 1 contents...
```

```
</FILE NAME='VTK file name 1'>\n
```

```
<FILE NAME= 'VTK file name 2'>\n
```

```
VTK file 2 contents...
```

```
</FILE NAME='VTK file name 2'>\n
```

```
...
```

- File names may contain any ASCII character allowed by VTK and the file system, other than single quote and NULL characters

Writing to a DPVTK File

- Open the file and create a raw output buffer

```
DPvzVtk* dpvz = new DPvzVtk(...);  
std::stringstream out;
```

- Record the VTK file header, contents, and footer

```
out << "<FILE NAME='" << "VTK file name" << "'>" << std::endl;  
out << VTK file contents...;  
out << "</FILE NAME='" << "VTK file name" << "'>" << std::endl;
```

- Write to the file

```
dpvz->write(cycle, time, out);
```

- Close the file

```
delete dpvz;
```

Notes and TODO Items

Architectural Limitations to DPvz File Size

- In the Time Step Table of Contents, the maximum number of cycles is 68,736,258,047 (68+ billion), which requires 38 bits to represent
- Time steps must be monotonic but need not be sequential
- In the Table of Contents (DPvzTocEntry):
 - Each entry points to one time step and contains:

```
int32_t  ranks;    // number of ranks participating in this time step
int32_t  cycle;    // simulation cycle of this time step
```
- `int32_t` can represent, at most, 2,147,483,648 (2 billion) cycles
- 2 billion cycles is well beyond projected use for the foreseeable future

Architectural Limitations (cont.)

- In order to enjoy the full limit of 68 billion cycles, the data types of `rank` and `cycle` in `DPvzTocEntry` must be set to `int64_t`
- Doing so will increase the `DPvzTocEntry` size from 32 to 40 bytes
- This will change the size of the table of contents (`DPvzToc`) from $3 \times 4,096 \times 32 = 393,216$ bytes (96 pages) to $3 \times 4,096 \times 40 = 491,520$ bytes (120 pages)
- Changing the size of `DPvzTocEntry` will render older DPvz files incompatible with newer versions of the DPvz library
- DONE • Change `write` and `truncate` function interfaces to `int64_t`, and add checks/warnings to parameter values

Architectural Limitations (cont.)

- Alternative Two might be to re-allocate the bits such that `ranks` is reduced to 24 bits (roughly 16 million ranks), and `cycle` is increased to 40 bits (well over 68 billion cycles)

```
int ranks:24;    // number of ranks in this time step
int cycle:40;    // simulation cycle of this time step
```

- This has the advantage of keeping the DPvz Table of Contents the same size as it is today (96 pages)
- Notes:
 - Files with this ToC layout are also incompatible with the current design
 - 16 million ranks is very large, but it will be reached long before 2 billion
 - Other size choices include `ranks:26` and `cycle:38`

File Distribution

- DONE** • Add a `chk_file_data_distribution` function to check whether to set the distribution when a new file is created
- DONE** • Support four values (not case sensitive) instead of two:
 - “file” to set the file distribution for each new file
 - “directory” to set the directory default distribution
 - “both” to set the file AND the directory distribution
 - “none” to NOT set either the file or directory distribution
 - Any other value, including `NULL` or “”, is the same as “none”

Default Stripe Size

- Default stripe size is set for very large-scale files
- Lower the default to suit more modest scales
- DONE • Change `DPVZ_DEFAULT_STRIPE_SIZE` to `1<<20`